

計算機程式設計

自主學習報告

主題：STL 與基礎資料結構入門

授課教師：林朝興教授

姓名：謝佳恩

學號：S11459020

自主學習主題與目標

自主學習的主題為，STL 與基礎資料結構入門，其目的為從「template 可以讓函式與類別套用於不同型別」的觀念，進一步認識 C++ 標準模板函式庫 STL，並熟悉 STL 容器，理解背後的基礎資料結構系統。

GitHub repo link 與檔案結構說明

<https://github.com/iliketotelltruth/C-Template-work.git>

檔案結構:README.md，c++_template.cpp，S11459020_謝佳恩自主學習報告.pdf。

Template 與 STL 的關係

模板(Template)是 C++ 中泛型編程(GP-Generic Programming)的基礎，而標準樣版函式庫(STL)則是泛型編程思想的成品，基於模板來實作各式各樣的資料結構與演算法，使 STL 可以處理任何特定需求的資料類型，並大幅簡化了 C++ 程式的開發，而組成 STL 的核心部分可分為，容器(Container)，演算法(Algorithms)，迭代器(Iterator)，仿函數(Functor)，適配器(Adaptor)。

常見 STL 容器用途整理

- 序列容器(Sequence Containers)
 - Vector: 動態陣列，支持快速隨機存取，vector 可在 $O(1)$ 內在末位插入/存取元素，但在中間或開頭則需 $O(n)$ 時間。
 - Deque: 與 vector 類似，能在常數時間內完成頭尾插入/刪除，但不支持隨機存取
 - List: 有序的資料結構，元素中存取著前後的位址，雙向的鏈結串列，與 vector 相比，元素訪問速度較慢，

在已知位址的情況下，插入與刪除速度較快。

- Forward_list: list 的單鏈結版，較省空間
- Array: 只能在初始化指定大小的陣列。
- 關聯容器 (Associative Containers)
 - Set: 不重複元素，自動排序。
 - Multiset: 與 set 類似，元素可重複。
 - Map: 關聯陣列，一個元素有兩個資料項，map 把一個資料對映到另一個
 - Multimap: 與 map 類似，但允許重複地鍵值。
- 無序關聯容器 (Unordered Associative Containers)
 - Unodered_set: 與 set 類似，但使用雜湊表實現。
 - Unodered_multiset: 與 multiset 類似，但使用雜湊表實現。
 - Unodered_map: 與 map 類似，但使用雜湊表實現。
 - Unodered_multimap: 與 multimap 類似，但使用雜湊表實現。
- 其他
 - Bitset: 儲存系列位類似的固定大小的布林向量。
 - Valarray: 數值類型的 vector，犧牲泛型能力為數值計算做了最佳化。

第二周小專題設計說明

使用 STL 管理學生資料，透過 vector 儲存學生資料，管理系統有基本的以下幾個能力，1. 新增學生資料，2. 列出所有學生，3. 依成績排序，4. 查詢學生，5. 統計成績，程式中也使用了兩個 Template function 來找出最高成績與最低成績，運用 do-while 迴圈和 switch 來完成資料系統，五個獨立的 function 分別達成不同功能。

主要程式架構

Mainfunction



```
int main() {
    vector<Student> students;
    int choice;

    do {
        cout << "\n Student Grade Management System\n";
        cout << "1. Add student\n";
        cout << "2. List all students\n";
        cout << "3. Sort by score\n";
        cout << "4. Search by id\n";
        cout << "5. Show statistics\n";
        cout << "0. Exit\n";

        if (!(cin >> choice)) {
            cin.clear();
            cin.ignore(10000, '\n');
            cout << "Invalid input. Please enter a number.\n";
            continue;
        }

        switch (choice) {
            case 1: addStudent(students); break;
            case 2: listAllStudents(students); break;
            case 3: sortByScore(students); break;
            case 4: searchById(students); break;
            case 5: showStatistics(students); break;
            case 0: cout << "Exiting program...\n"; break;
            default: cout << "Invalid choice. Please try again.\n";
        }
    } while (choice != 0);

    return 0;
}
```

```
template <class T>
T getMax(T a, T b) {
    return (a > b) ? a : b;
}

template <class T>
T getMin(T a, T b) {
    return (a < b) ? a : b;
}
```

template function

```
struct Student {
    string id;
    string name;
    int score;
};

void addStudent(vector<Student>& students);
void listAllStudents(const vector<Student>& students);
void sortByScore(vector<Student>& students);
void searchById(const vector<Student>& students);
void showStatistics(const vector<Student>& students);
```

功能函式宣告，學生結構宣告。

執行畫面或輸出結果

學生資料系統起始畫面

```
Student Grade Management System
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
```

加入學生資料

```
C:\Users\k2006\source\repos\ x + v
4. Search by id
5. Show statistics
0. Exit
1
Enter ID: 250
Enter Name: roger
Enter Score: 83
Student added successfully.

Student Grade Management System
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
1
Enter ID: 8
Enter Name: nl
Enter Score: 8
Student added successfully.

Student Grade Management System
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
|
```

展示全部學生資料，排序學生資料後展出學生資料

```
C:\Users\k2006\source\repos\ x + v
ID      Name      Score
-----
114514  haotwo    19
250     roger     83
8       nl        8

Student Grade Management System
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
3
Students sorted by score (descending).

ID      Name      Score
-----
250     roger     83
114514  haotwo    19
8       nl        8

Student Grade Management System
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
|
```

查找學生 ID:114514，學生統計圖表

```
C:\Users\k2006\source\repos\ x + v
5. Show statistics
0. Exit
4
Enter ID to search: 114514
Student found: 114514 | haotwo | Score: 19

Student Grade Management System
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
5

--- Statistics ---
Class Average: 36.67
Highest Score: 83
Lowest Score: 8
Passed (>=60): 1
Failed (<60): 2

Student Grade Management System
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
|
```


遇到的問題與解決方式

template 和 STL 都是在學校學習過，但實際使用的次數極為稀少，對於詳細的使用方法還不太熟悉，因此在自主學習的過程中，也在網路上查找了各式各樣的資料，閱讀了許多程式設計前輩的部落格內容，這些都有助於我對於這次自主學習有更加深刻的幫助。

心得

這次程式碼實作裡使用了 STL 容器當中的 vector，我個人選擇使用 vector 的原因是我在各式各樣 STL 容器中，對 vector 的熟悉度是最高的，我在許多程式碼中都使用過這個容器，在學習了 STL 和 template 後，在做專題上面能在面對不同條件下，選擇最好的程式去實作，不論是自己製作專題，又或是與同伴們一同製作，都能成為我對於程式設計的修習的養分，希望未來能有更多這樣的機會。